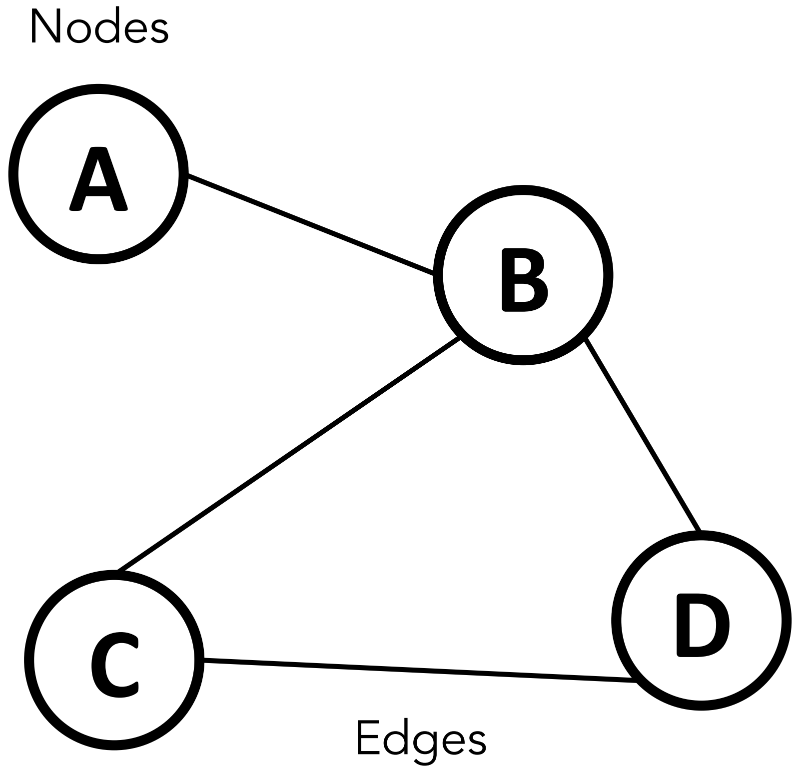
Graph Neural Networks

Tri Nguyen (CIERA Fellow)

NASA AI/ML STIG | Feb 02, 2026

What is a graph?



A graph is a mathematical data structure that represents relationships between different nodes

Graph is all you need

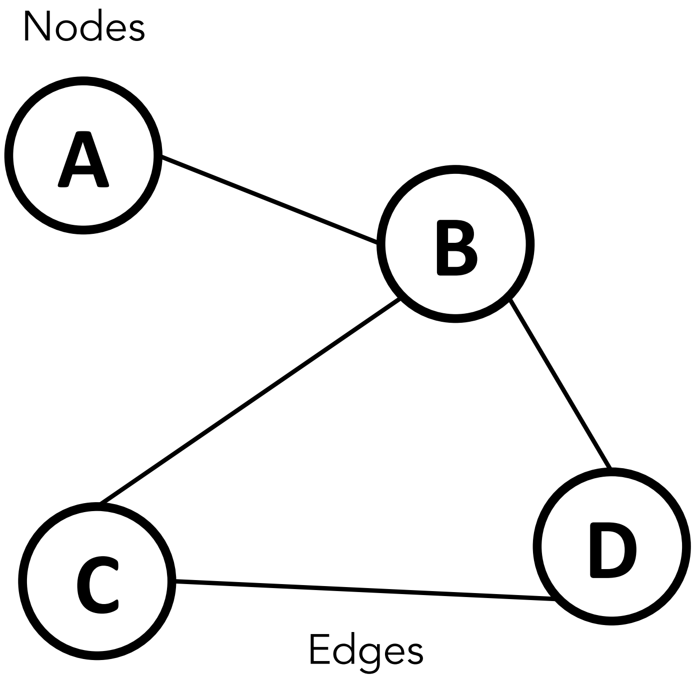
Graph structure can represent everything and is everywhere, all at once

molecular structures

traffic & subway network

social network

What is a graph?



A graph is fully described by:

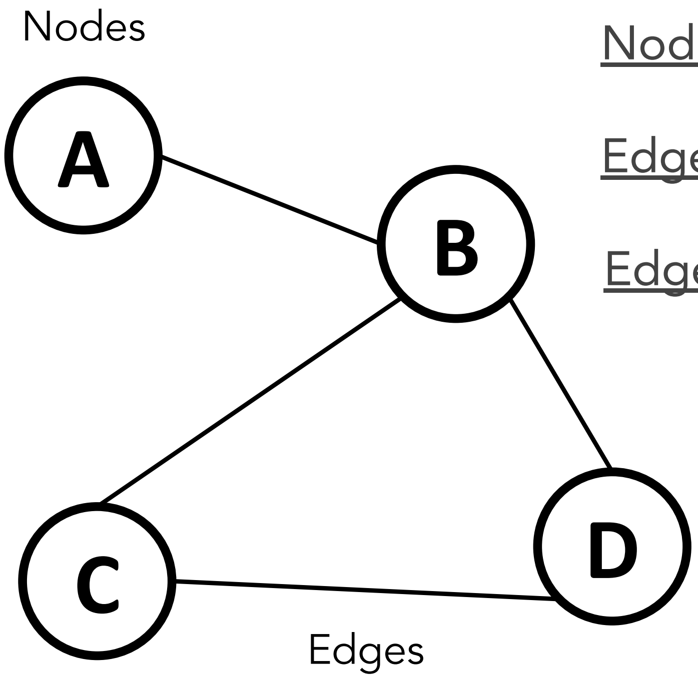
Node set: $\mathcal{V} = \{v_1, v_2, ..., v_n\}$

Edge set: $\mathcal{E} \subseteq \{(u, v) : u, v \in \mathcal{V}\}$

It is often more useful to express the edge set as an *adjacency matrix*

$$A_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \in \mathcal{E} \\ 0 & \text{otherwise} \end{cases}$$

What is a graph?



Optionally, we can also have:

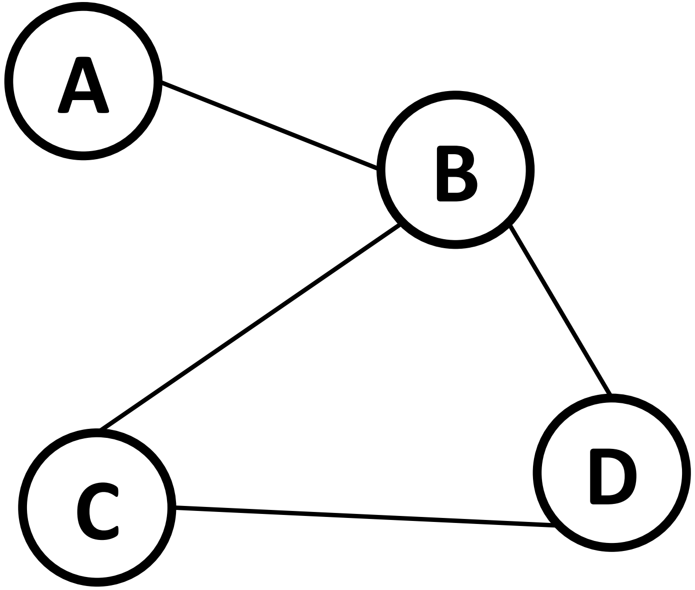
Node features: $\mathbf{X}_{\text{node}} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$

Edge features: $\mathbf{X}_{\text{edge}} = \{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_{|\mathcal{E}|}\}$

Edge weights: $\mathbf{w} = \{w_1, w_2, \ldots, w_{|\mathcal{E}|}\}$

And any global graph features (e.g. number of loops)

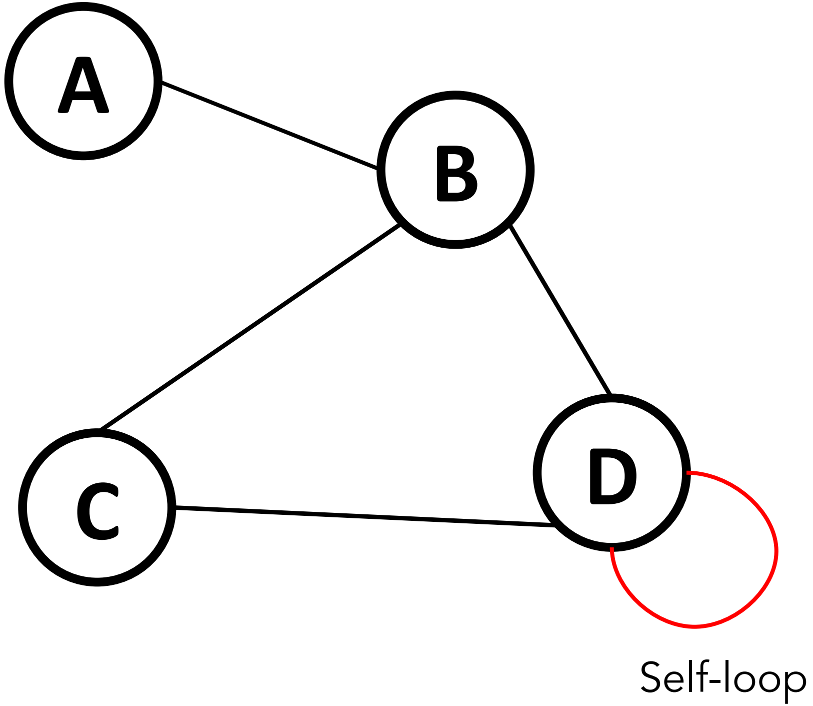
Adjacency matrix



$$\begin{array}{c c} & \begin{array}{cccc} A & B & C & D \end{array} \\ \begin{array}{c} A \\ B \\ C \\ D \end{array} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

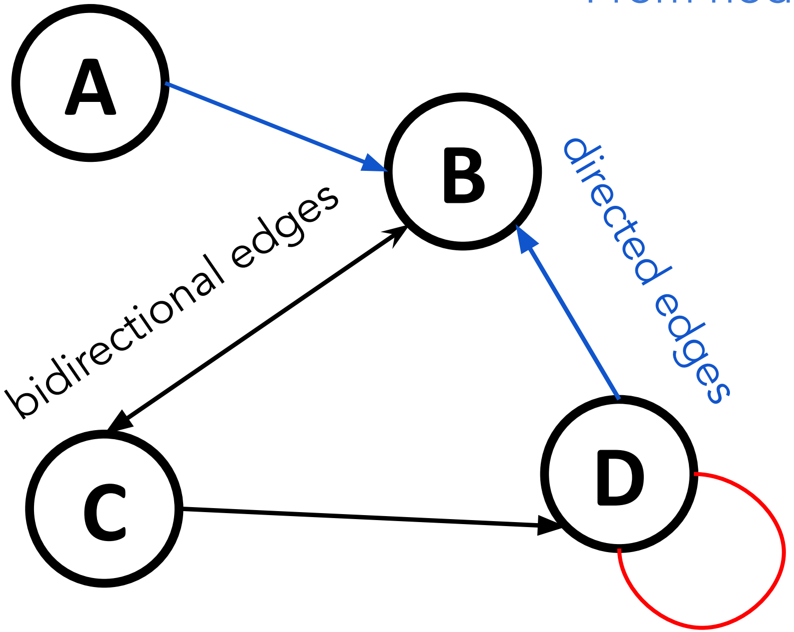
Adjacency matrix

A graph can have self-loops, which connect nodes to themselves



$$
\begin{array}{c c}
 & \begin{array}{cccc} A & B & C & D \end{array} \\
\begin{array}{c} A \\ B \\ C \\ D \end{array} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}
\end{array}
$$

Adjacency matrix



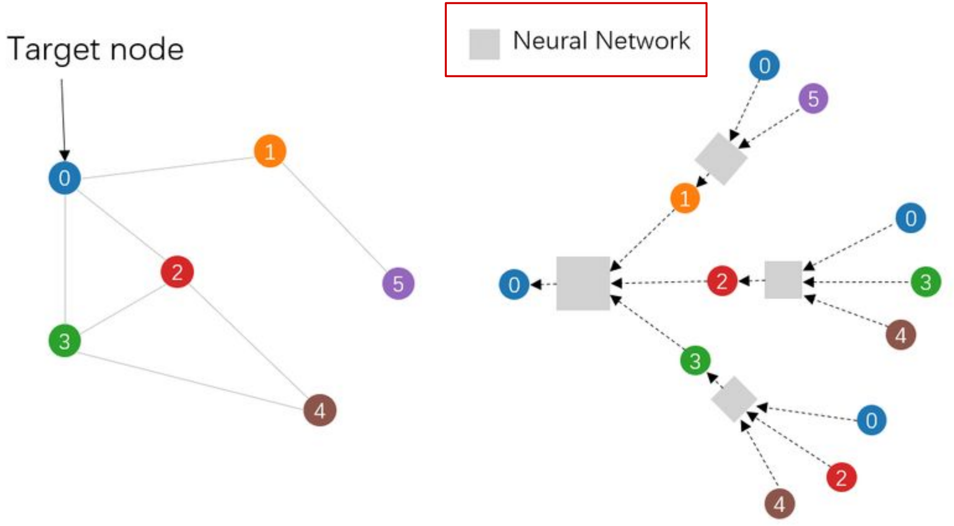
A graph can be directed or undirected

From node *i* to node *j* : row *i* & column *j*

$$\begin{array}{c c} & \begin{array}{cccc} A & B & C & D \end{array} \\ \begin{array}{c} A \\ B \\ C \\ D \end{array} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix} \end{array}$$

Graph neural network (GNN)

GNNs learn node embeddings through iterative *message passing*, where each layer aggregates and transforms features from neighboring nodes

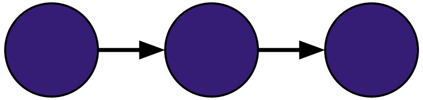


(a) Input graph

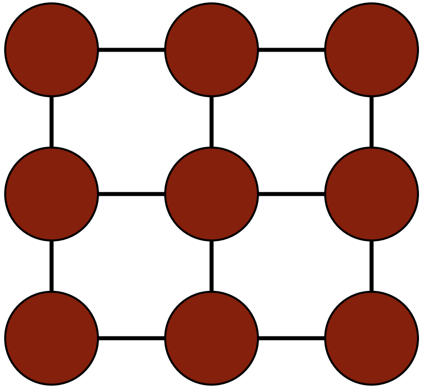
(b) Neighborhood aggregation

Connection to previous data structures

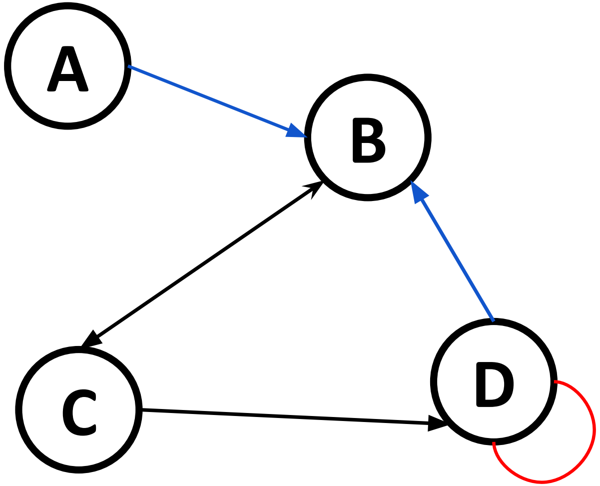
Time Series



Images



Euclidean space



Non-Euclidean space

Connection to previous data structures

Image

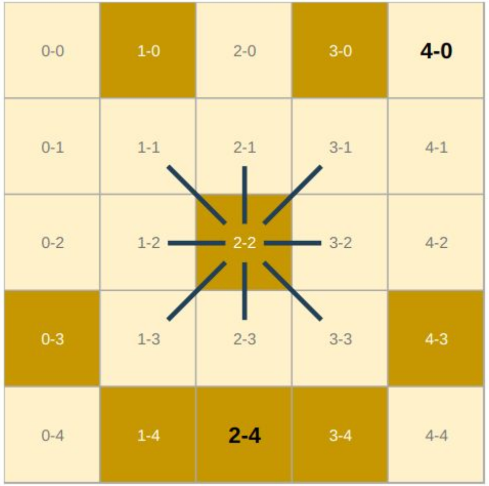
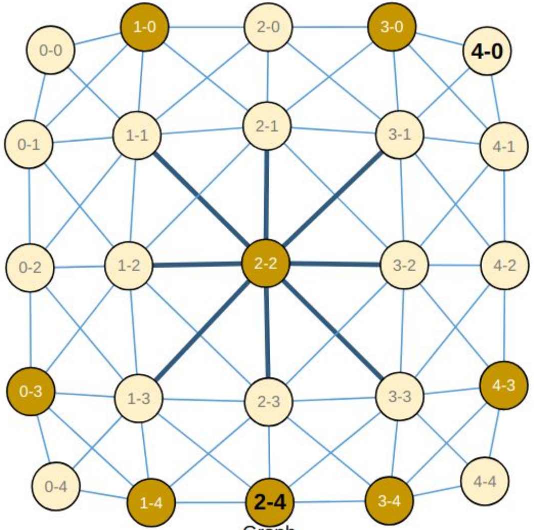


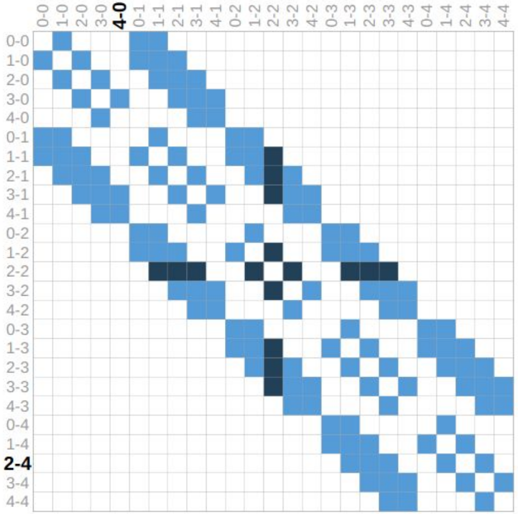
Image Pixels

Graph



Graph

Adjacency Matrix



Adjacency Matrix

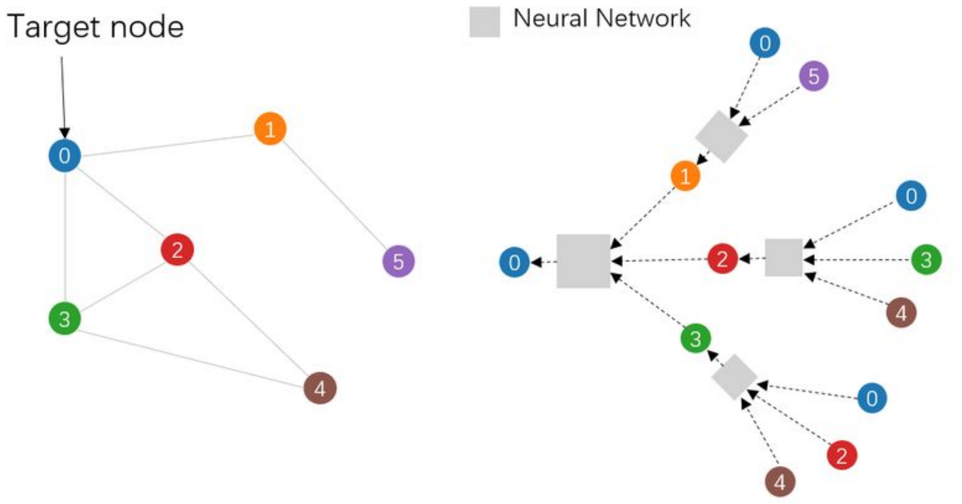
https://distill.pub/2021/gnn-intro/

Message passing

$$\mathbf{h}_i^{(l+1)} = \text{UPDATE}^{(l)}\left(\mathbf{h}_i^{(l)}, \text{AGGREGATE}^{(l)}\left(\{\mathbf{h}_j^{(l)} : j \in \mathcal{N}(i)\}\right)\right)$$

Message passing aggregates features from neighboring nodes through learned transformations

This includes self-loops



(a) Input graph (b) Neighborhood aggregation

Example: Graph Convolutional Network (GCN)

Aggregation operation: $$\mathbf{h}_i^{(l+1)} = \sigma\left(\mathbf{W}^{(l)} \sum_{j \in \mathcal{N}(i)} \frac{\mathbf{h}_j^{(l)}}{|\mathcal{N}(i)|}\right)$$

In matrix form:

$$\mathbf{H}^{(l+1)} = \sigma\left(\mathbf{D}^{-1}\mathbf{A}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)$$

where D is the degree matrix (with self-loops): $$D_{ii} = \sum_j A_{ij}$$

Kipf & Welling 2016
arXiv:1609.02907

Example: Graph Attention Network (GAT)

Vaswani et al. (2017)
Attention Is All You Need

Aggregation operation: $$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(l)} \mathbf{W}^{(l)} \mathbf{h}_j^{(l)}\right)$$

where the attention coefficient is:

$$\alpha_{ij}^{(l)} = \frac{\exp\left(\mathrm{LeakyReLU}\left(\mathbf{a}^{(l)T}[\mathbf{W}^{(l)}\mathbf{h}_i^{(l)} \| \mathbf{W}^{(l)}\mathbf{h}_j^{(l)}]\right)\right)}{\sum_{k\in\mathcal{N}(i)} \exp\left(\mathrm{LeakyReLU}\left(\mathbf{a}^{(l)T}[\mathbf{W}^{(l)}\mathbf{h}_i^{(l)} \| \mathbf{W}^{(l)}\mathbf{h}_k^{(l)}]\right)\right)}$$

<u>NOTE</u>: No matrix form exists for GAT

See next week's lecture by Helen Qu

Veličković et al. (2018)
arXiv:1710.10903

Types of GNN tasks

	Supervised	Unsupervised
Node-level	Node classification	Node clustering
Edge-level	Link prediction, edge classification	Edge embedding, graph reconstruction
Graph-level	Graph classification	Graph embedding, graph generation

Known Issues in GNNs

<u>Oversmoothing</u>: Deep GNNs cause node features to converge (all features look the same after a certain depth)

=> limiting depth to ~3-5 layers for most architectures

<u>Over-squashing</u>: Information from distant nodes gets compressed through narrow bottlenecks

=> limiting long-range dependencies

Math: The Graph Laplacian

We can also define a Laplacian matrix:

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

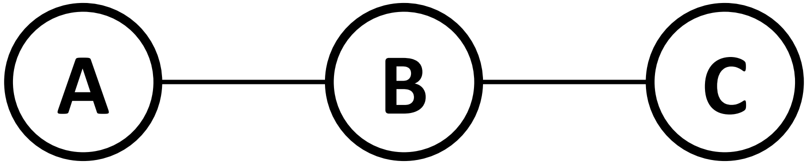
Two common normalized forms of the Laplacian:

$$\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$$

$$\mathbf{L}_{\text{rw}} = \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$$

Math: The Graph Laplacian

The Laplacian measures the ***local variations*** or ***smoothness*** of signals in a graph



$$\frac{\mathrm{d}^2 f}{\mathrm{d}x^2} = \lim_{h \to 0} \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

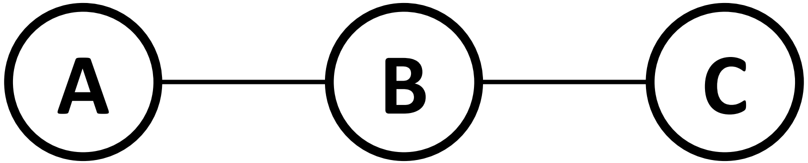
Laplacian

Some features

$$\begin{bmatrix} 1 & \text{-}1 & 0 \\ \text{-}1 & 2 & \text{-}1 \\ 0 & \text{-}1 & 2 \end{bmatrix} \quad \mathbf{x}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad \mathbf{L}\mathbf{x}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Math: The Graph Laplacian

The Laplacian measures the <u>*local variations*</u> or <u>*smoothness*</u> of signals in a graph



$$\frac{\mathrm{d}^2 f}{\mathrm{d}x^2} = \lim_{h \to 0} \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

Laplacian

Some features

<u>Notice the sign difference</u>

$$\begin{bmatrix} 1 & \text{-}1 & 0 \\ \text{-}1 & 2 & \text{-}1 \\ 0 & \text{-}1 & 2 \end{bmatrix} \qquad \mathbf{x}_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \qquad \mathbf{L}\mathbf{x}_2 = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

Math: Spectral Decomposition

We can decompose the Laplacian into ***eigenvectors*** and ***eigenvalues***, which represent the ***modes*** and their corresponding ***frequencies*** of the graph

$$\lambda_1 = 0 \qquad \lambda_2 = 2 \qquad \lambda_3 = 4$$

$$\mathbf{u}_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad \mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \quad \mathbf{u}_3 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

Math: Spectral Decomposition

GNNs generalize convolution from regular grids to arbitrary graph structures

GCN aggregation operator:

$$\mathbf{H}^{(l+1)} = \sigma\left(\mathbf{D}^{-1}\mathbf{A}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)$$

A symmetrized form of the Laplacian:

$$\mathbf{L}_{\text{rw}} = \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$$

Some guidelines when using GNNs

Graph construction matters:

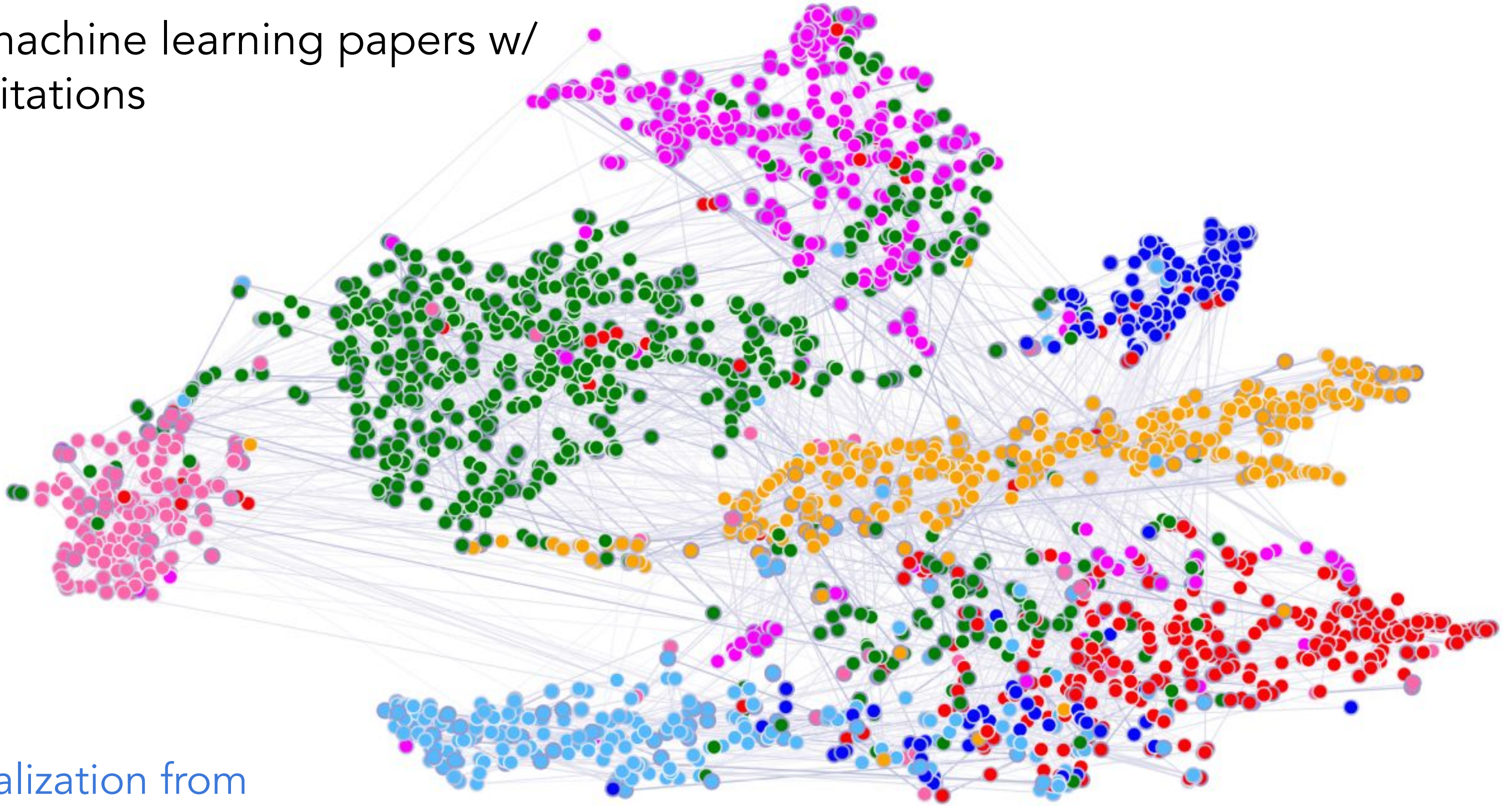
- Local vs global structure
- Graph type: k-NN, radius, learned edges (e.g. GAT)
- Generalization to out-of-domain data

Common GNN Types:

- Fixed aggregation: GCN, ChebConv
- Learned aggregation: GAT, GraphSAGE, GraphTransformer
- Hybrid: GraphGPS (local message passing + global attention)

Tutorial 1: Cora citation network - Node classification

2,708 machine learning papers w/
5,429 citations



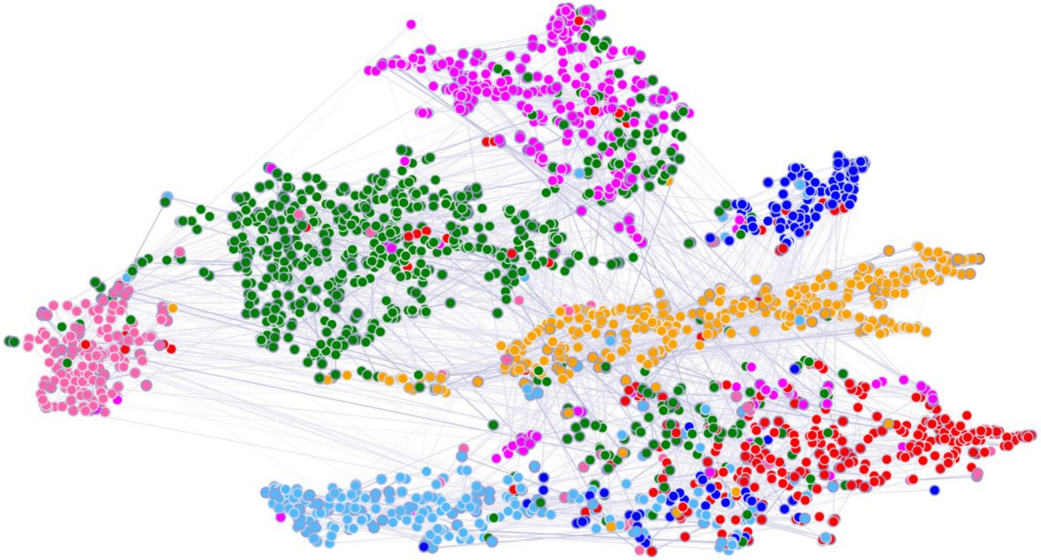
t-sne visualization from
GAT paper

Tutorial 1: Cora citation network - Node classification

Each paper is encoded by a binary word vector of 1,433 unique words

Classes: Case Based, Genetic Algorithms, Neural Networks Probabilistic Methods, Reinforcement Learning, Rule Learning, Theory

Goal: train a GNN model to classify paper based on the binary word vectors of their citations



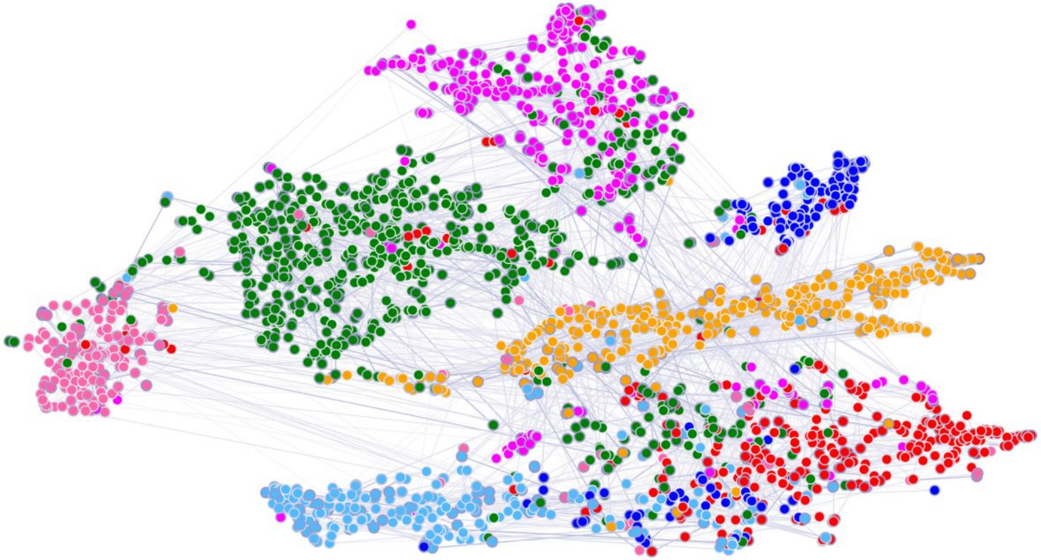
Tutorial 1: Cora citation network - Node classification

The dataset includes ONE giant graph, split into 4 sets:

1. Training nodes: 140
2. Validation nodes: 500
3. Test nodes: 1000
4. Unlabeled nodes: the rest

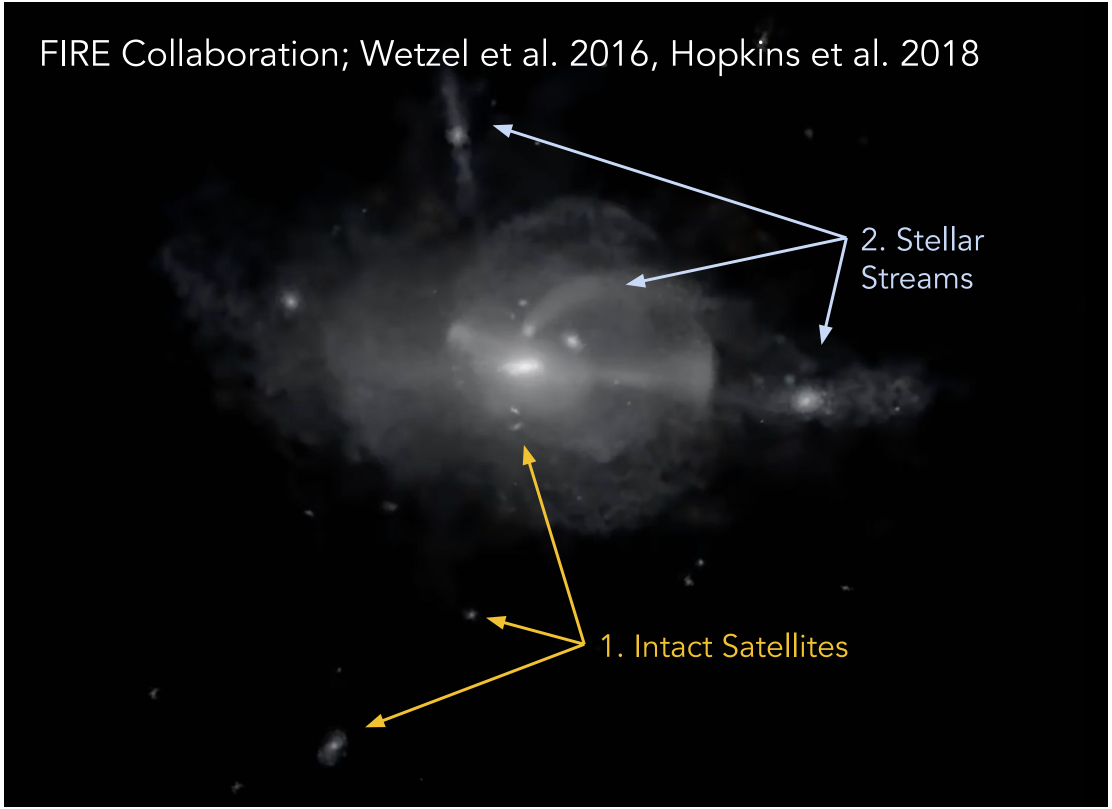
The unlabeled nodes *are still being used* during message passing, just not in the loss calculation

This can be done by masking nodes appropriately



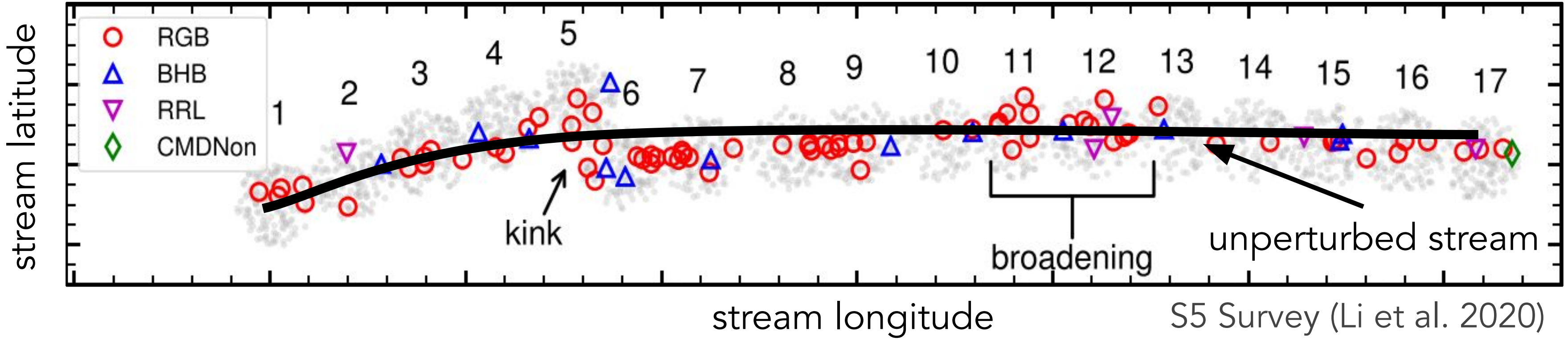
Tutorial 2: Stellar stream - Graph regression

Stellar streams are tidal remnants of dwarf galaxies and globular clusters



Tutorial 2: Stellar stream - Graph regression

Dark matter subhalos can create density perturbations in stellar streams, which can be used to infer the subhalo properties



S5 Survey (Li et al. 2020)

GD-1 stream: Hilmi et al. (2024)

Atlas Aliqa-Uma: Nguyen et al. (2025)

Tutorial 2: Stellar stream - Graph regression

The dataset includes multiple stellar streams

Input: 6D coordinates of tracer stars in streams. Each can be constructed into a single graph

Goal: recover dark matter subhalo parameters:

- Subhalo mass
- Subhalo velocity